ROADMAP DE DESARROLLO & ARQUITECTURA

PLAN DETALLADO DE CONSTRUCCIÓN FASE 2

Yireh Store — Portal Web Angular, POS, BI, Supabase & Despliegue Cloud

Especificación técnica exhaustiva de requerimientos pendientes, arquitectura web administrativa en **Angular**, módulo de **Cajero/POS**, analítica de ventas, **migración a Supabase**, microservicios de correo y **dockerización integral**.

ALCANCE DE LOS REQUERIMIENTOS A CONSTRUIR:

- **Frontend Web Administrativo (Angular):** Panel web corporativo para gestión centralizada, monitoreo comercial y administración financiera.
- **Módulo de Punto de Venta (POS / Cajero):** Sistema de cobro ágil en mostrador con soporte de efectivo, tarjeta y débito de billetera institucional.
- **Inteligencia de Negocio & Finanzas:** Historial de ventas por día/mes, arqueo y cierre diario de caja, y proyecciones de demanda.
- **Inventario de Activos Físicos:** Control de máquinas, cafeteras, refrigeradores, depreciación y mantenimiento preventivo.
- **Encuestas de Satisfacción (CSAT / NPS):** Calificación post-pedido en app móvil y métricas de servicio.
- **Servicio de Email Transaccional:** Envío de facturas digitales en PDF y extractos mensuales de compras por cliente vía SMTP/API.
- **Migración a Supabase:** Transición de MySQL local a PostgreSQL gestionado en la nube con Row Level Security.
- **Dockerización & Servidor Web:** Contenedores Docker multi-stage para Frontend, Backend y despliegue en VPS Cloud con HTTPS.

Documento: Especificación de Requerimientos y Plan de Acción | **Proyecto:** Yireh Store | **Versión:** 2.0 (Planificación) | **Fecha:** Septiembre 2026

1. Vistas Administrativas en Angular (Web Dashboard)

Actualmente, la administración se gestiona parcialmente desde la app móvil en Flutter. Para optimizar las operaciones de gerencia, contabilidad y atención en caja, se especifica la construcción de un **Frontend Web dedicado en Angular** (v17/v18 Standalone Components) con diseño corporativo tipo dashboard, soporte para teclado numérico de caja y visualización gráfica avanzada.

Arquitectura Propuesta para Angular

```
angular-admin-yireh/
├── src/
│   ├── app/
│   │   ├── core/                  # Guards (Auth, Role), Interceptors (JWT), Servicios Base
│   │   ├── layout/                # Sidebar, Topbar, Notificaciones, Footer
│   │   ├── modules/
│   │   │   ├── pos/               # Punto de Venta (Cajero) con escaneo y cobro rápido
│   │   │   ├── analytics/         # Ventas diarias, mensuales, KPIs y proyecciones
│   │   │   ├── cash-closure/      # Arqueo de caja, diferencias y cierre de turno
│   │   │   ├── assets/            # Inventario de maquinaria, mobiliario y mantenimientos
│   │   │   ├── satisfaction/      # Reportes de encuestas y opiniones de clientes
│   │   │   ├── products-catalog/  # CRUD extendido de categorías y productos
│   │   │   └── email-center/      # Reenvío de facturas y auditoría de envíos
│   │   └── shared/                # Tablas dinámicas, exportador Excel/PDF, modales
```

Módulos Clave del Dashboard Angular

🖥️ Punto de Venta (POS / Cajero)

Interfaz optimizada para pantallas táctiles y teclado. Búsqueda instantánea de productos, botón de cobro rápido con efectivo, tarjeta o escaneo del código QR del estudiante para débito de saldo en billetera.

📊 Ventas & Proyecciones

Gráficas interactivas (Chart.js / ngx-charts) con comparativas de ventas diarias y mensuales, productos más rentables, horarios de mayor afluencia y modelos de proyección predictiva.

💰 Arqueo & Cierre de Caja

Registro de base inicial en efectivo, conteo de billetes/monedas, contraste contra ventas registradas por sistema y generación del acta formal de cierre diario.

⚙️ Inventario de Activos Físicos

Fichas técnicas de equipos (cafeteras industriales, molinos, congeladores, terminales), registro de valor de compra, depreciación acumulada y fechas de mantenimiento.

2. Módulos de Operación Financiera (Cajero & Cierres)

2.1. Módulo de Cajero / Punto de Venta (POS)

El objetivo de este módulo es atender a los clientes que realizan compras directamente en el mostrador de la cafetería sin usar la aplicación móvil, manteniendo el inventario y las finanzas completamente sincronizados con las órdenes móviles.

Requerimiento	Detalle Funcional	Criterio de Aceptación
Búsqueda & Selección	Selección rápida por grid de fotos, categorías o lector de código de barras.	La adición al ticket de venta debe tomar menos de 1 segundo.
Multi-Método de Pago	Pago combinado: Efectivo + Saldo Billetera Estudiante + Tarjeta/Datafono.	El sistema calcula automáticamente el cambio o saldo faltante.
Débito QR Institucional	El cajero escanea el código de usuario del estudiante para debitar de su saldo.	Requiere validación de PIN o confirmación rápida en pantalla.
Impresión / Envío de Ticket	Generación de factura POS en ticketera térmica (80mm) o envío al email del cliente.	Se emite documento electrónico con consecutivo fiscal.

2.2. Historial de Ventas, Cierre de Caja y Proyecciones

Actualmente el sistema registra pedidos individuales pero carece de consolidación contable y proyecciones. Se requiere implementar en el Backend y exponer en Angular:

- **Cierre Diario de Caja (Z-Report):**
  - Apertura de turno con monto base.
  - Separación estricta de ingresos: *Efectivo en gaveta* vs *Saldo virtual debitado* vs *Datáfono*.
  - Cálculo de descuadre: `Diferencia = Monto Físico Declarado - Monto Sistema`.
  - Bloqueo de modificaciones sobre pedidos cerrados tras el cierre del turno.
- **Ventas por Día y Mes:** Consolidado con filtros de fecha, categorización de costos y margen de utilidad bruta.
- **Proyecciones de Demanda:** Modelo estadístico basado en series temporales (promedios ponderados por día de semana y eventos institucionales) para sugerir compras de materias primas a los proveedores.

2.3. Control e Inventario de Activos Físicos

Campo / Dato	Descripción	Impacto Contable
`asset_code`	Placa única de inventario físico (código de barras/código QR metálico).	Identificación y auditoría física.
`name / category`	Ej: Cafetera Espresso 2 Grupos, Molino On-Demand, Congelador Vertical.	Agrupación de activos fijos.
`purchase_price / date`	Costo original de adquisición y fecha de inicio de uso.	Base contable para amortización.
`maintenance_schedule`	Calendario de mantenimientos preventivos (cambio de empaques, descalcificación).	Prevención de paradas operativas en cocina.

3. Experiencia de Usuario & Servicios de Correo

3.1. Servicio de Email Transaccional (Spring Mail / SendGrid)

El sistema actual no cuenta con notificaciones salientes. Se debe construir un micro-servicio desacoplado de correo en el backend Spring Boot utilizando `JavaMailSender` o una API externa (como SendGrid/Resend) con plantillas HTML responsivas para las siguientes acciones:

📧 Facturas Digitales Automáticas

Al confirmar una orden (en app o en POS), se genera automáticamente un PDF con la factura detallada y se remite al correo registrado del comprador.

📑 Historial & Extracto Mensual

Función bajo demanda y programada (`@Scheduled`) para enviar a fin de mes un extracto de todos los consumos, recargas y saldo remanente al cliente o a sus acudientes.

💳 Comprobante de Recarga

Confirmación inmediata de recargas exitosas indicando código de referencia bancario, monto acreditado y saldo actualizado de la billetera.

🔔 Alerta de Cierre Diario a Gerencia

Envío nocturno del resumen consolidado del cierre de caja al administrador con las ventas totales y discrepancias de dinero en caja.

3.2. Módulo de Encuestas de Satisfacción al Cliente (CSAT / NPS)

Para medir la calidad del producto y del servicio en la cafetería, se diseñará un flujo interactivo en la aplicación móvil y en el portal web:

- **Disparador (Trigger):** Se activa automáticamente cuando el pedido cambia al estado `DELIVERED` (Entregado).
- **Métricas Recopiladas:**
  - Calificación por estrellas (1 a 5 estrellas).
  - Tags rápidos: "*Temperatura perfecta*", "*Rápida entrega*", "*Buen sabor*", "*Atención amable*" o "*Demorado*".
  - Campo opcional de sugerencia de texto libre.
- **Dashboard de Métricas en Angular:** Visualización del índice de satisfacción global, quejas recurrentes y ranking de productos mejor valorados.

Nuevos Endpoints a Desarrollar en el Backend

Endpoint	Método	Propósito	Rol
`/api/pos/checkout`	POST	Registro de venta física directa en caja mostrador	CASHIER / ADMIN
`/api/cash-closures/open`	POST	Apertura de turno con monto base en efectivo	CASHIER / ADMIN
`/api/cash-closures/close`	POST	Cierre formal de caja diario con arqueo de dinero	CASHIER / ADMIN
`/api/reports/sales`	GET	Ventas agregadas por rango de fechas (día/mes)	ADMIN
`/api/reports/projections`	GET	Cálculo predictivo de demanda para la siguiente semana	ADMIN
`/api/assets`	GET / POST	CRUD y control de inventario de activos de cocina	ADMIN

Endpoint	Método	Propósito	Rol
`/api/satisfaction/surveys`	POST	Envío de calificación y comentarios de una orden	USER
`/api/emails/send-invoice/{orderId}`	POST	Disparo manual o reintento de envío de factura PDF	ADMIN

4. Infraestructura Cloud: Supabase, Docker & Despliegue

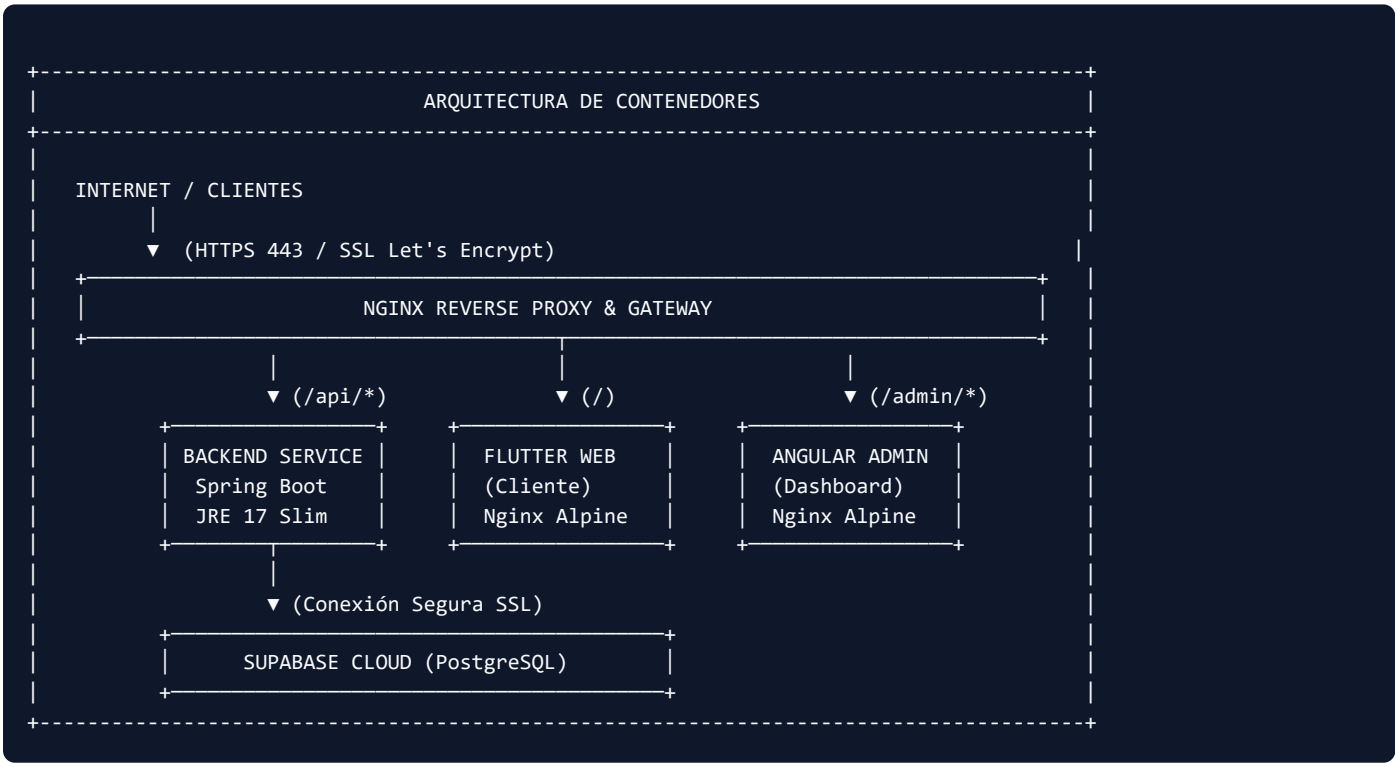
4.1. Migración de Base de Datos: MySQL ➔ Supabase (PostgreSQL)

Actualmente el backend utiliza MySQL local. Migrar hacia **Supabase** proporciona una base de datos PostgreSQL gestionada de alto rendimiento, respaldos automatizados, alta disponibilidad, extensiones avanzadas y capacidades en tiempo real (*Realtime WebSockets*) ideales para el tablero de cocina.

> **Plan de Migración Paso a Paso:**
> 1. **Creación del Proyecto en Supabase:** Creación de la instancia PostgreSQL y obtención del URI de conexión JDBC con SSL (`jdbc:postgresql://db.[ref].supabase.co:5432/postgres?sslmode=require`).
> 2. **Adaptación de Dependencias Maven:** Reemplazar `mysql-connector-j` por `org.postgresql:postgresql` en el `pom.xml` del backend.
> 3. **Ajuste de Dialecto JPA:** Configurar `spring.jpa.properties.hibernate.dialect = org.hibernate.dialect.PostgreSQLDialect` en `application.properties` .
> 4. **Conversión de Tipos DDL:** Mapear tipos MySQL (`INT AUTO_INCREMENT` ➔ `BIGSERIAL` , `DATETIME` ➔ `TIMESTAMPTZ` , `TINYINT(1)` ➔ `BOOLEAN`).
> 5. **Migración de Datos Existentes:** Ejecución de script de exportación MySQL y carga en Supabase preservando relaciones e integridad referencial.

4.2. Estrategia de Dockerización Multi-Contenedor

Para garantizar portabilidad y despliegue continuo sin discrepancias de entorno, se crearán contenedores multi-stage optimizados:



4.3. Especificación de Dockerfiles y Docker Compose

- **Backend Dockerfile (Spring Boot):** Multi-stage con etapa de compilación en `maven:3.9-eclipse-temurin-17` y etapa de ejecución ligera en `eclipse-temurin:17-jre-alpine` (reducción de tamaño de ~700MB a ~140MB).
- **Angular Dockerfile:** Compilación con `node:20-alpine` y servido final mediante `nginx:alpine` con compresión Gzip activada.

- **Flutter Web Dockerfile:** Servido de los activos compilados de Flutter Web en Nginx para acceso en navegadores.
- **Docker Compose (`docker-compose.yml`):** Orquestación unificada con reinicio automático (`restart: always`), redes internas aisladas y montaje de variables de entorno (`.env`).

4.4. Despliegue en Servidor Web Cloud (VPS)

- **Proveedor Recomendado:** Servidor VPS Linux (Ubuntu 22.04 / 24.04 LTS) en DigitalOcean, AWS EC2, Hetzner o Render.
- **Seguridad & Certificados:** Configuración de **Certbot / Let's Encrypt** para emisión y renovación automática de certificados SSL/TLS con candado verde seguro.
- **Dominio y DNS:** Configuración de registros tipo A y CNAME (`yirehstore.com` para la tienda y `admin.yirehstore.com` para la administración en Angular).

5. Plan de Ejecución y Cronograma de Trabajo

Para asegurar una entrega ordenada y sin interrupciones en la operación actual, se establece un plan de trabajo dividido en **5 Fases estructuradas**:

Fase	Hito / Entregable	Tareas Principales	Prioridad
Fase 1	**Infraestructura & Supabase**	• Aprovisionar proyecto en Supabase. • Migrar schema y scripts SQL a PostgreSQL. • Actualizar driver en Spring Boot y validar conexión.	CRÍTICA
Fase 2	**Servicios Core de Backend**	• Crear endpoints para POS y apertura/cierre de caja. • Diseñar modelos de datos de Activos Físicos y Encuestas. • Implementar micro-servicio de Email y generación de PDF.	CRÍTICA
Fase 3	**Frontend Angular (Dashboard & POS)**	• Inicializar proyecto Angular con componentes modulares. • Desarrollar interfaz táctil del Cajero / POS. • Construir gráficas de ventas, proyecciones y cierre de caja. • Crear vistas para inventario de activos y encuestas.	ALTA
Fase 4	**Integración en Flutter Móvil**	• Incorporar formulario de calificación post-entrega. • Generación de código QR para débito de saldo en caja mostrador. • Botón de "Enviar extracto de compras a mi correo".	MEDIA
Fase 5	**Dockerización & Despliegue Cloud**	• Construir Dockerfiles multi-stage y `docker-compose.yml`. • Configurar VPS en la nube y Nginx Reverse Proxy. • Instalar certificados SSL gratuitos con Let's Encrypt. • Pruebas integrales de carga y paso a producción.	ALTA

Matriz de Responsabilidades y Roles Técnicos

Rol	Responsabilidad Principal en la Fase 2
Backend Engineer	Adaptación a Supabase, endpoints POS, reportes de ventas, lógica de cierre de turno y templates de email.
Frontend Web (Angular)	Construcción de dashboard gerencial, punto de venta táctil, visualización de gráficas y gestión de activos.
Frontend Mobile (Flutter)	Módulo de encuestas CSAT, vista de código de barras/QR de billetera y reenvío de comprobantes por email.
DevOps / Cloud	Creación de imágenes Docker, orquestación, configuración de dominio DNS, Nginx y hardening del servidor VPS.

Conclusión y Recomendación

La ejecución de este plan transformará a **Yireh Store** en una plataforma omnicanal integral de nivel empresarial. Al sumar el punto de venta en mostrador, el control riguroso de caja e inventario, la analítica predictiva y una infraestructura en la nube contenerizada con Supabase, el sistema garantizará alta fiabilidad, automatización contable y una experiencia de compra superior tanto física como digital.

Aprobación Técnica

Arquitectura de Software Yireh Store

Aprobación de Proyecto

Comité de Desarrollo & Operaciones